4.

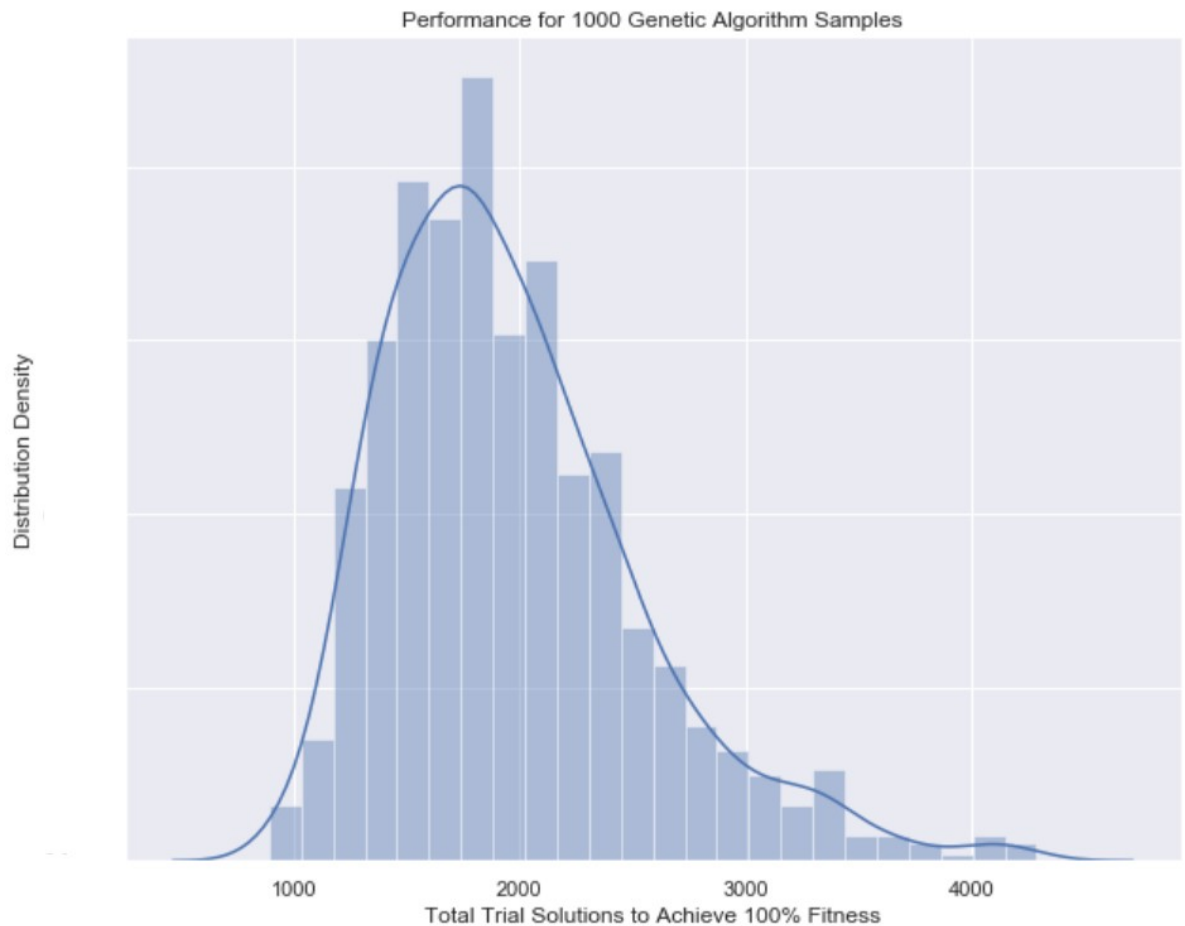
The Schema Theorem

The general presentation follows the monograph by Michalewicz

©FII 2020-2021

- Inspired from Nature, Genetic Algorithms have a behaviour of their own
- Which laws describe the macroscopic behaviour of Genetic Algorithms?
- How do low-level operators (selection, crossover, mutation) result in the macroscopic behaviour of Genetic Algorithms?
- What makes a problem suited for GAs?
- What makes a problem unsuited for GAs?
- Which performance criteria are relevant?
- Top=level reference: "*Foundations of Genetic Algorithms*" series (started in 1991).

An Example of a Theoretical Question for GA



©Adam C. Dick,, Univ.of Michigan

Typical behaviour of a GA: distribution, over 1000 runs, of the number of chromosomes generated before reaching the global optimum (for the *Battleship* game).

*Does the GA follow a **GAMMA distribution** of the convergence speed? Why?*

Another Example of a Theoretical Question for GA

```
procedure Augmented GA
begin
        t = 0;
        initialize P(t);
(*)     Q(t) = complement(P(t);
        evaluate structures in P(t);
        while termination condition not satisfied do
        begin
                t = t + 1;
                select P(t) from P(t-1);
                alter structures in P(t);
(*)             Q(t) = complement(P(t));
                evaluate structures in P(t);
(*)             evaluate structures in Q(t);
                output best structure in P(t) ∪ Q(t);
        end
end.
```

Augmented GA was proposed for avoiding *deception* (John J. Grefenstette - *Deception considered harmful*, 2nd F.O.G.A. Conf., Vail, Colorado). ***Is it worth using it?***

Existing Theoretical Models

- **Schema Theorem**:
  - "The two-armed bandit problem" (Holland 1975);
  - "Royal Roads" (Forrest, Holland, Mitchell, 1992)
- **(Exact) mathematical models** of simple GAs (Vose – Markov chains – 1991, Goldberg 1987, Davis 1991, Horn 1993, Whitley 1993)
- **Statistical-Mechanics** approaches (Prügel-Bennett 1994)

The Schema Approach

- Holland builds the Schema Theorem apparatus in the frame of the **standard Genetic Algorithm**:
  - bitstring genotype of fixed length, **m**
  - one-point crossover
  - standard mutation (bit negation)
  - *fortune wheel* selection procedure
- The abstract model focuses on the search space of the **genotype**
  - It relates to the **phenotype** only through fitness.
- *Schema* (plural: *schemata*) helps formalise an informal notion: “building blocks”.

Schema

- In the search space of representations of candidate solutions, a schema <u>is</u> **a set** of bitstrings (a hyperplane) which <u>is described</u> by **a template** made up of ones, zeroes and asterisks (wild cards).
- Examples for **m**=10.

•Example 1.

– as a **set**: H_1={(0011000111), (0011000110)}

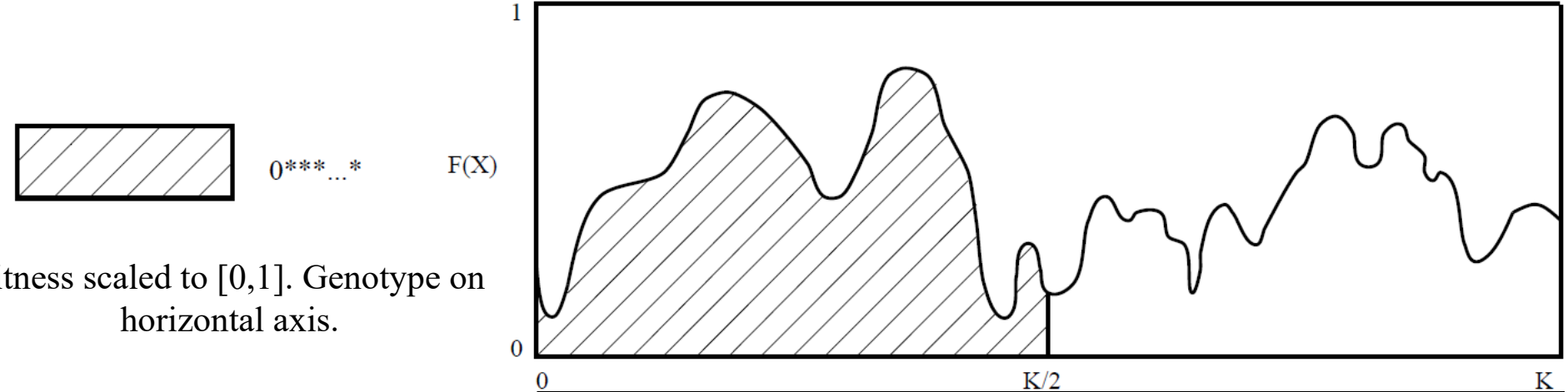
– as **template**: S_1=(001100011*)

•Example 2.

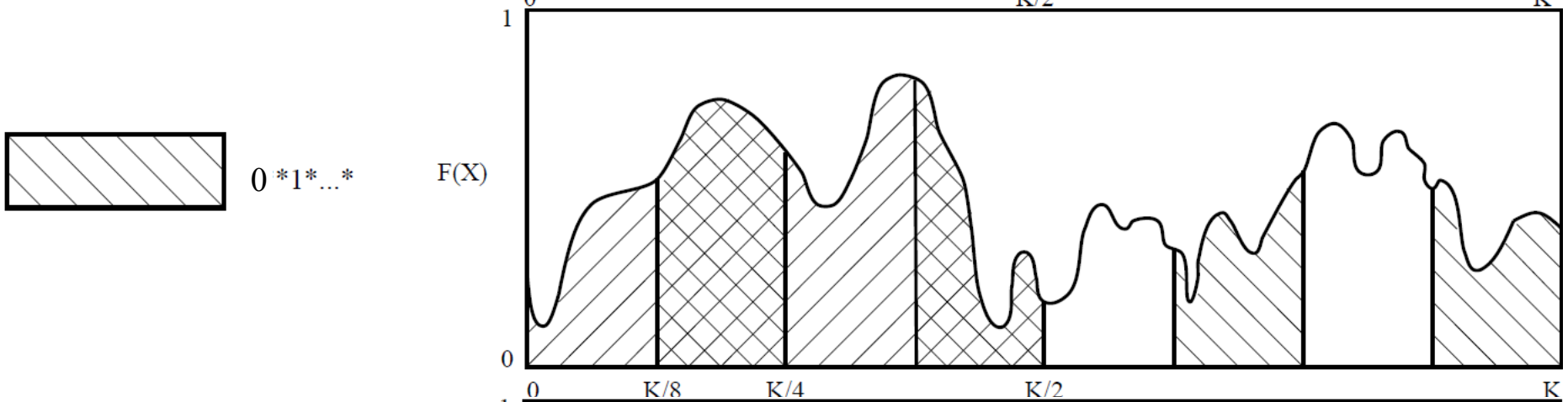
– **Set**: H_2={(0011000111), (0001000111), (0011001111), (0001001111)}

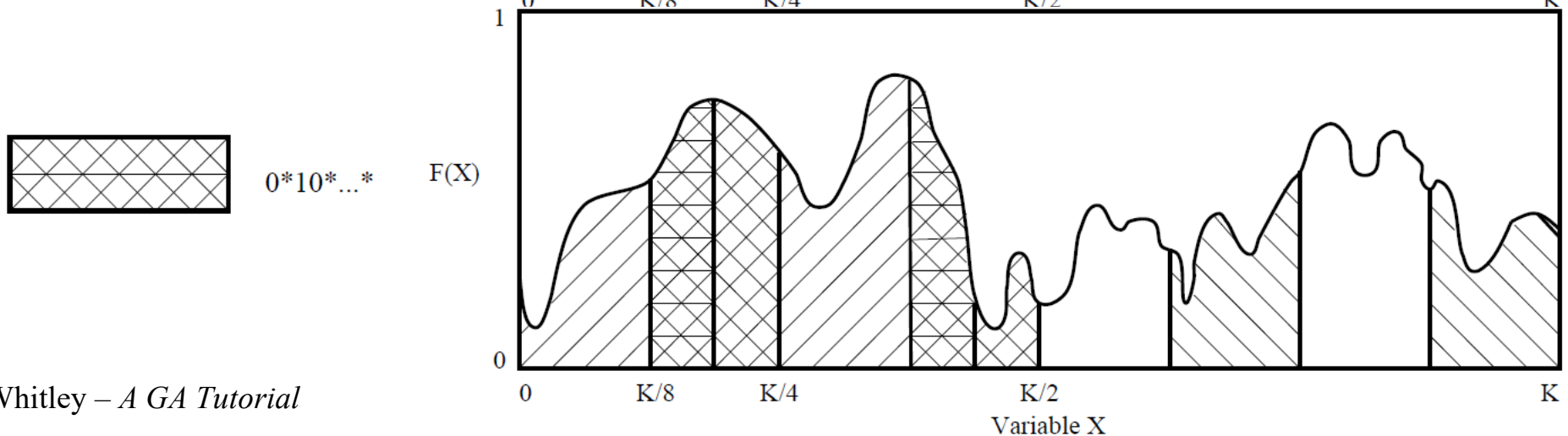
- **Template**: S_2=(00*100*111)

In the sequel, we shall use interchangeably the word *schema* for both the set and the template.



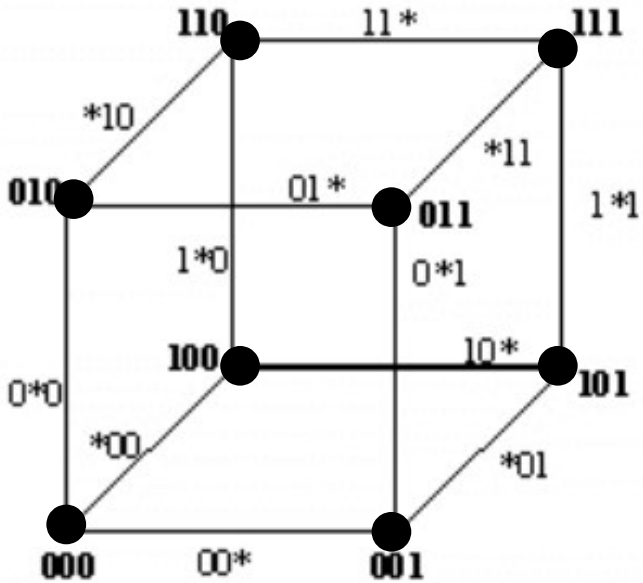
Fitness scaled to [0,1]. Genotype on horizontal axis.



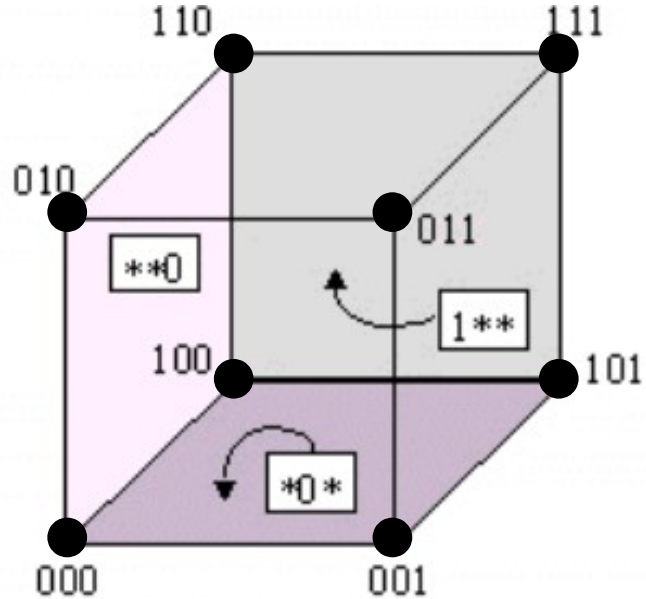


©D.Whitley – *A GA Tutorial*

Schema: Set and Template



Search space for a problem with L=3



Three of the six schemata of order 1

Order of the Schema o(H)	Hyperplane Dimension	Geometric Interpretation	Strings in the Schema	Number of Schemata
3	0	Point	1	8
2	1	Line	2	12
1	2	Plane	4	6
0	3	Cube	8	1
		Discrete, not continuous	**Total**	**27**

©University of Malaga

Schemata as templates

- A schema with *a* wildcards describes a hyperplane (set) containing 2^a binary strings.
- A chromosome (binary string of length *m*) is matched by / is represented by / **is an instance** of, 2^m schemata.
  - m=2. **01** is an instance of: **01**, **0***, ***1**, ****.
- For a given *m*, there exist 3^m schemata
    - and 2^m chromosomes.
- Between 2^m and $pop_size \cdot 2^m$ of them may be represented in one generation.
    - The two bounds correspond to a) a generation consisting only of copies of one chromosome and b) all chromosomes in the generation are different.
- QUIZ (this time, for schemata as sets): When is the union / difference of two schemata, also a schema ?

Characteristics of Schemata as Templates

1. **Defined bits of a schema S**: all non-wildcard positions of S.
2. **Order of a schema S, o(S)**:

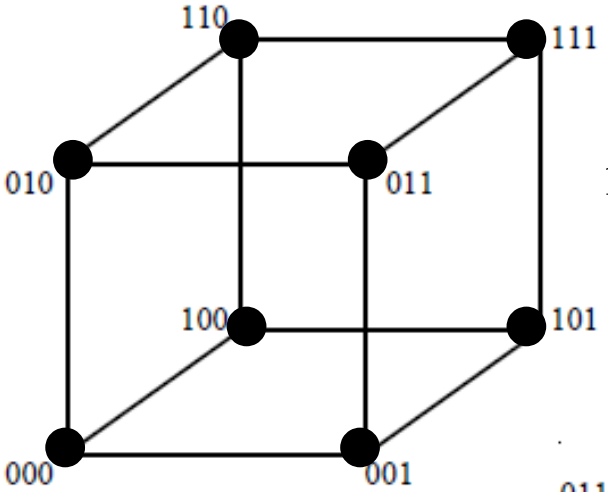
   number of defined bits in S

   - how specific is S?
   - In example 2 above, S_2=(00*100*111), $o(S_2) = 8$.
3. **Defining length of a schema S, δ(S)**: the distance between its outermost <u>defined</u> bits.
   - how compact is the information in the schema?
   - In example 2, $\delta(S_2) = 10-1 = 9$.
   - if S has only one defined bit, then $\delta(S)=0$.

Schema H	$o(H)$	$\delta(H)$	
1********	1	0	0 points where crossover could distroy the schema
011*	3	2	2 breaking points
*1*11***	3	3	3 breaking points

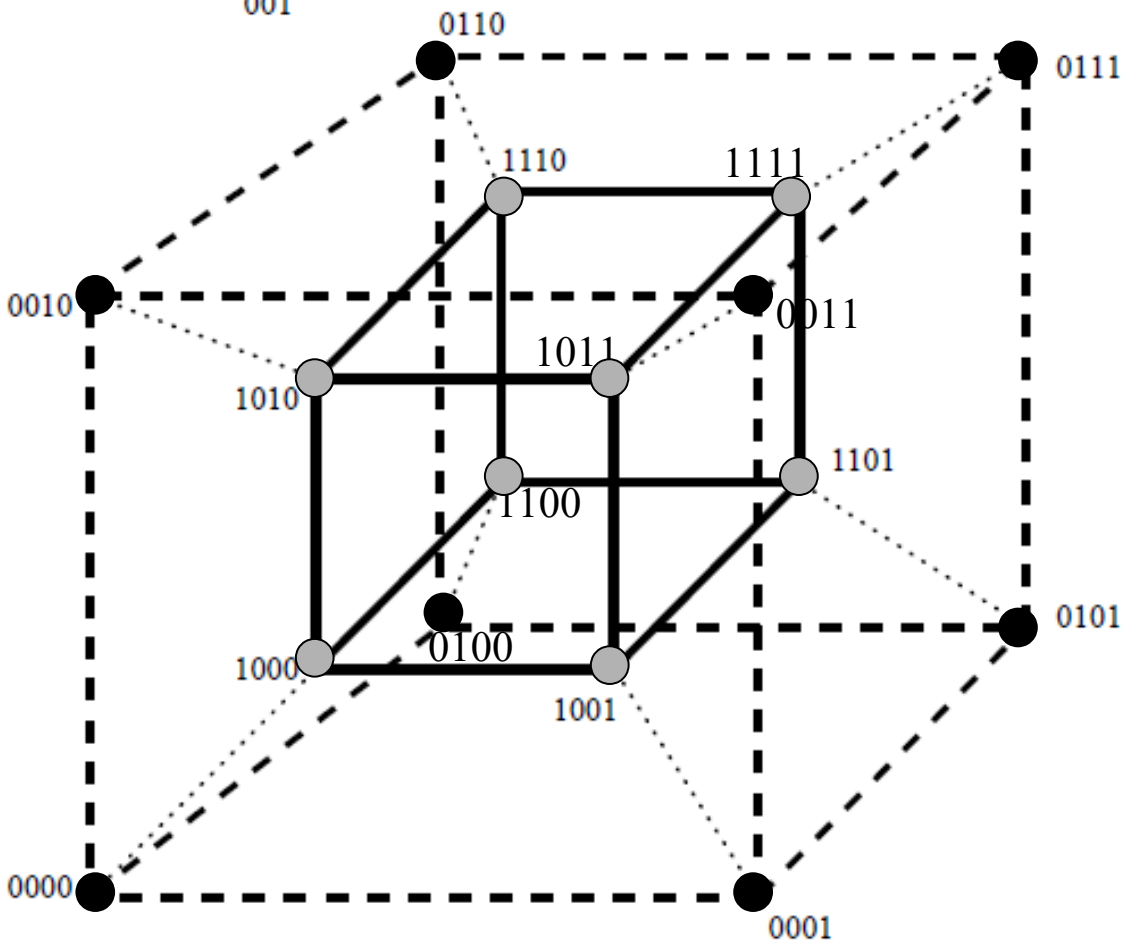
©University of Malaga

QUIZ: 3-dim cube / 4-dim cube



m=3

m=4



Vertices are order ? schemata
Edges are order ? schemata
Planes are order ? schemata
Cubes (a type of hyperplane)
are order ? schemata

Hint: 8 different order-1 schemata:
0***, 1***, *0**,
*1**, **0*, **1*, ***0, ***1

©D.Whitley – *A GA Tutorial*

Characteristics of Schemata (2)

4. Static average fitness of a schema S in a Genetic Algorithm: the arithmetic mean of the evaluations of all instances str_i of S (**static fitness**):

$$\boldsymbol{eval}(\mathrm{S}) = (1/2^a)\cdot\sum_{i=1..2^a} \boldsymbol{eval}(str_i)$$

- a – the number of wildcards in S.

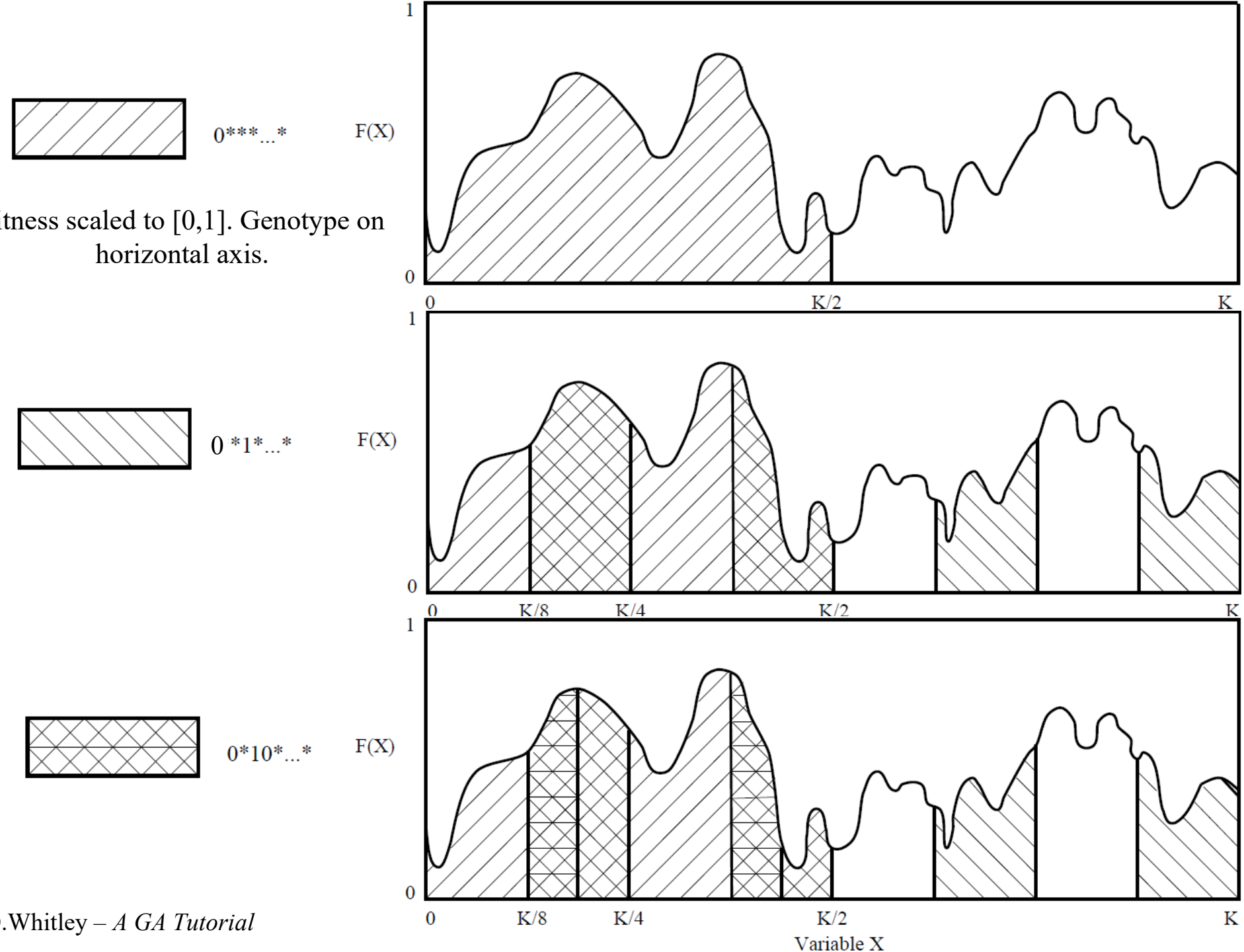
5. *Absolute* average fitness of a schema S in generation t of a specific Genetic Algorithm (**dynamic-fitness**):

$$\boldsymbol{eval}(\mathrm{S},t) = (1/k)\cdot\sum_{j=1..k} \boldsymbol{eval}(str_j,t)$$

k – the number of instances str_j of S, in generation P(t).

If k=0, then $\boldsymbol{eval}(\mathrm{S},t)$=0.

***Relative* dynamic fitness**: **eval**(S,t)/ $\bar{f}(t)$, the dynamic fitness divided by the average fitness of the whole generation. $\bar{f}(t)=\Sigma_{i=1...pop_size}\mathbf{eval}(\mathrm{str}_i(t))/\mathrm{pop_size}$



Fitness scaled to [0,1]. Genotype on horizontal axis.

©D.Whitley – *A GA Tutorial*

Evaluation of Schemata

- Schemata **are not** explicitly represented / evaluated by the GA.
- Estimates of the dynamic fitness for schemata **are not** calculated / stored by the GA.
- However, <u>the number of instances</u> of given schemata in successive generations increases or decreases, **as if the dynamic fitness of schemata were actually calculated**!

- This is what the schema theorem proves (see next slides).

Schema Theorem: a study on schema dynamics

- Notation: "$x \in H$" for "x is an instance of H"
- Let **H** be a schema with

  $\boldsymbol{\eta}(\mathbf{H},\mathbf{t})$ instances (chromosomes representing H) in generation **t**;

  eval(H,t) - the observed average fitness of H in generation **t.**

$$\mathbf{E}(\boldsymbol{\eta}(\mathbf{H},\mathbf{t}+1)) = ?$$

(what is the *expected number* of instances of **H** in generation **t**+1 ?)

The Effect of Selection

- One chromosome **x** will have in the next generation an expected number of **eval(x)** / $\bar{f}(t)$ offspring.

  f(x) = **eval** (**x**); $\bar{f}(t)$ - average fitness in generation **t**

$$E(\eta(H,t+1)) = \sum_{x \in H} \text{eval}(x)/\bar{f}(t) = \eta(H,t) \cdot \text{eval}(H,t)/\bar{f}(t),$$

since $\text{eval}(H,t) = \left(\sum_{x \in H} \text{eval}(x)\right)/\eta(\text{H,t})$.

- The GA does not calculate explicitly **eval(H,t)**, however this quantity "decides" the number of instances of **H** in subsequent generations.
- *Schemata with higher dynamic-fitness will have more chrmosomes representing them in the next generation*

Schemata and Fitness Values										
Schema	Mean	Count	Expect	Obs		Schema	Mean	Count	Expect	Obs
101*...*	1.70	2	3.4	3		*0**...*	0.991	11	10.9	9
111*...*	1.70	2	3.4	4		00**...*	0.967	6	5.8	4
1*1*...*	1.70	4	6.8	7		0***...*	0.933	12	11.2	10
01...*	1.38	5	6.9	6		011*...*	0.900	3	2.7	4
**1*...*	1.30	10	13.0	14		010*...*	0.900	3	2.7	2
11...*	1.22	5	6.1	8		01**...*	0.900	6	5.4	6
11**...*	1.175	4	4.7	6		0*0*...*	0.833	6	5.0	3
001*...*	1.166	3	3.5	3		*10*...*	0.800	5	4.0	4
1***...*	1.089	9	9.8	11		000*...*	0.767	3	2.3	1
0*1*...*	1.033	6	6.2	7		**0*...*	0.727	11	8.0	7
10**...*	1.020	5	5.1	5		*00*...*	0.667	6	4.0	3
*1**...*	1.010	10	10.1	12		110*...*	0.650	2	1.3	2
****...*	1.000	21	21.0	21		1*0*...*	0.600	5	3.0	4
						100*...*	0.566	3	1.70	2

Relative dynamic fitness (*Mean*) for the 27 schemata (*Schema*) given by the first 3 bits. **For each schema**, the columns give the expected number of chromosomes in the next generation (*Expect*), the actual number in the current generation (*Count*) and the actual number after selection for the next generation (*Obs*).

©D.Whitley – *A GA Tutorial*

The Effect of Crossover

- Effect: destroy or create instances of **H**.
- Consider the *disruptive effect* only → lower bound for $\mathbf{E}(\eta(\mathbf{H},\mathbf{t}+1))$.
- Suppose an instance of **H** is a parent.

If H=10***...******, $\delta(\mathbf{H})=1$, the probability of disruption is **1/m-1**

If H=1*****...****0, $\delta(\mathbf{H})=\mathbf{m-1}$, the probability of disruption is **(m-1)/(m-1)=1**

- The lower bound for the probability $s_c(\mathbf{H})$ that **H** will survive after crossover:

$$s_c(\mathbf{H}) \geq 1 - p_c \cdot (\delta(\mathbf{H})/(m-1))$$

- *The probability of survival under crossover is higher for more compact ("shorter") schemata.*

The Effect of Mutation

- Again, consider the *destructive effect* only
- What is the probability that no defined bits will be mutated?

  If H=**1**01*1****, o(H)=4, survival-after-mutation probability is $s_m(H)=(1-p_m)^4$

- The probability of survival under mutation:

$$s_m(\mathbf{H}) = (1-p_m)^{o(\mathbf{H})}$$

- *The probability of survival under mutation is higher for lower-order schemata.*

Schema Theorem

hypothesis

In a Genetic Algorithm with bitstring chromosomes of length *m*, flip-bit mutation with rate p_m, one-point crossover with rate p_c and fortune wheel selection,

a schema *H* of order o(*H*), defining length δ(*H*), with eval(H, t) absolute dynamic fitness in generation t, and which is represented by η(*H*, t) chromosomes in generation t

conclusion

will have in generation t+1 a lower bound for the **expected** number of chromosomes representing it, η(*H*, t+1), given by the formula:

$$E\left(\eta\left(H\quad,t+1\right)\right)\geq\frac{\mathrm{eval}\left(\mathrm{H,t}\right)}{\bar{f}\left(t\right)}\cdot\eta\left(\mathrm{H,t}\right)\cdot\left(1\text{-}p_c\cdot\frac{\delta\left(H\right)}{\mathrm{m\text{-}1}}\right)\cdot\left(1\text{-}p_m\right)^{o\left(H\right)}$$

- The theorem describes the **minimal** expected growth of the number of chromosomes representing a schema, from **one generation to the next**.

A Consequence of the Schema Theorem

- One way to extrapolate the Schema Theorem:

- "Short,
  low-order,
  constantly above average schemata
  receive **exponentially increasing** numbers of instances (chromosomes) in successive generations of the Genetic Algorithm."

- because the - supposedly constant - factor of growth is eval(H,t)/$\bar{f}(t)$
  (see next slides)

  - Note that the generation average fitness $\bar{f}(t)$ also increases in successive generations!

Exponential growth of Building Blocks:

- Assume *H* is a short, low-order, highly fit schema
- $f(H) > \overline{f}$ assumed: $f(H) = \overline{f} + c$
- Assume *H* is not destroyed by crossover or mutation
- Assume this remains valid for a number of generations

©Thomas Back, Leiden University

Exponential growth of Building Blocks (cont.):

$$\eta(H,t+1) = \eta(H,t) \cdot \frac{\bar{f}+c}{\bar{f}} = \eta(H,t) \cdot (1+c')$$

For a number of generations this means:

$$\eta(H,t) = \eta(H,0) \cdot (1+c')^t$$

(Similarly, if H has chromosomes in the population only from generation k)

$\rightarrow$ Exponential growth of H in the population!

In a Genetic Algorithm, η(H,t) grows exponentially for very few schemata. Indeed, since $\bar{f}(t)$ is almost always growing, the dynamic fitness of the schema should grow even faster, which is possible for very few schemata.

©Thomas Back, Leiden University

Building Blocks Hypothesis

The Building Blocks Hypothesis: A genetic algorithm evolves representations of (near-)optimal solutions by juxtaposing ***building blocks*: short, low-order, highly fit schemata**

If this is true, then **representation** is crucial: there should exist small groups of genes which represent useful features of optimal solutions.

If such groups exist, then the GA performs well.

A Building Block represents a large, promising sub-space of the search space.

- Large: low-order schema
- Promising: constantly above average fitness

The use of fixed-length chromosomes is implied by the BBH.

- Intuitively, variable length chromosomes sample different search spaces
- For example, in the Optimal Sorting Networks problem, the search space of candidate sorting networks with 60 comparators, the search space of sorting networks with 61 comparators etc.

Building Block identification

Bit Position :	1	2	3	4	5	6	7	8	9	10
Chromosome 1 (C1) :	1	0	1	1	0	1	0	1	0	1
Chromosome 2 (C2) :	1	1	1	1	0	0	1	0	0	1
Common Schema :	1	*	1	1	0	*	*	*	0	1
Building Block :	$\mathbf{B_1}$			$\mathbf{B_3}$						$\mathbf{B_5}$
Uncommon Schema C1:	*	0	*	*	*	1	0	1	*	*
Building Block :		$\mathbf{B_{2,1}}$					$\mathbf{B_{4,1}}$			
Uncommon Schema C2:	*	1	*	*	*	0	1	0	*	*
Building Block :		$\mathbf{B_{2,2}}$					$\mathbf{B_{4,2}}$			

©Chalermsub Sangkavichitr, Prabhas Chongstitvatana : *The Use of Explicit Building Blocks in Evolutionary Computation,* International Journal of Systems Science, 2016.

Operators as "Builders"

- **Crossover** is considered a major source of GA power.
  - recombines instances of good schemas to create instances of (oftenly enough) at least that good schemas
- **Building Block Hypothesis**: the *supposition* that this is how GAs work.
    - However, it depends on **representation** and operators
- **Mutation** provides *diversity* even when the population tends to converge
  - if in the population a bit becomes definitely 0, then only the mutation gives a chance to still try 1.

Implicit Parallelism

- In one generation, a Genetic Algorithm acts as if it:
  - estimates the average fitnesses of **all schemata** which have instances in that generation;
  - increases / decreases the representation of **these** schemata in the next generation accordingly.
- **Implicit evaluation of a large number of schemata** using only *pop_size* chromosomes.
  - Climbing many hills in parallel ("The GA as a massively parallel hillclimber")
- This is **implicit parallelism** (Holland 1975).
- Different from *inherent parallelism*
  - GAs lend themselves to parallel implementations
    - e.g., the evaluation of chromosomes

Implicit Parallelism (2)

In a generation of **k** different chromosomes of length **m**

$$\mathbf{2^m} \leq \text{number of schemata processed} \leq \mathbf{k*2^m}$$

$\mathbf{k^3}$ of which are processed usefully (Holland 1989)

Meaning:

- $\mathbf{k^3}$ schemata are not disrupted by crossover and mutation
- selection and operators bias the search towards better schemata (by the Schema Theorem)

As the GA proceeds, **k** decreases: less and less different chromosomes/schemata are present in one generation

- The GA focuses the search on promising hyperplanes (schemata)

Selection in Action

- Selection **biases** the sampling towards instances of (**more and more defined**) schemata whose average fitness is estimated to be above the generation's average.
  - This is the only guidance towards better solutions.
- As the population average increases due to selection, the schemata with better and better average fitness have higher and higher orders.
  - Getting rid of weak sub-schemata.
  - There are however counterexamples (so-called "deceiving problems" – see the Messy Genetic Algorithm, next file).

Adaptation Revisited

- Holland: an adaptive system should **identify, test** and **incorporate** structural properties which are likely to give better performance in some environment.
- **Exploration vs. Exploitation**:
  - search for new, useful adaptations vs.
  - use and propagation of adaptations.
    - Mutation rate; selection procedure; pop_size...
- Neglecting one of them may lead to:
  - **overadaptation** (inflexibility to novelty) "stuck"
    - Premature convergence (convergence to sub-optimal solutions)
  - **underadaptation** (few/no gained properties)
    - Closer to random search.
- A good GA properly balances exploration and exploitation.

The *Two-Armed Bandit* (2AB) Problem

- A model in statistical decision theory, adaptive control (Bellman 1961).
- Holland (1975) used it to study how Genetic Algorithms allocate samples (chromosomes) to schemas.
- A gambler has N coins to play a two-armed slot machine. The goal is to maximize the N-times payoff.
  - the two arms are labeled A_1 and A_2;
  - their mean payoffs-per-trial are μ_1 and μ_2;
  - which are stable over the experiment (stationary, independent processes);
  - μ_1 and μ_2 are unknown to the player;
  - she can only estimate them by playing coins in each arm.

2AB Is Useful for Genetic Algorithms

- How should coins be allocated to each arm, given:
  - the observed payoffs
  - the respective estimates of the average payoffs μ_1, μ_2 ?
- **Resource allocation under uncertainty.**
- Not for **guessing** the better arm, but rather for **maximizing** the payoff (on-line!)
- It is a combined goal, which could be losely described as: "find the solution using given, fixed resources".
- Holland: 3^m schemata → 3^m arms. Strategy: allocate exponentially more trials (chromosomes) to the "better" arms (schemata).
- Maximisation of on-line performance of GAs.

The frame for solving the 2AB Problem

- Assume $\mu_1 \geq \mu_2$: A_1 **has** higher average payoff than A_2.
- After N trials, let:
  - $A_H(N, N\text{-}n)$ be the arm with **observed** higher payoff;
    - » not necessarily A_1
    - » may switch between A_1 and A_2 during the N trials, in which case A_L switches simultaneously
  - $A_L(N, n)$ be the arm with **observed** lower payoff;
    - » not necessarily A_2
    - » may switch between A_1 and A_2 during the N trials, in which case A_H switches simultaneously.
  - n be the number of trials allocated to $A_L(N, n)$;
  - N-n – the number of trials allocated to $A_H(N, N\text{-}n)$.
- Problem statement: *Find n^* to maximize expected profits over N trials.*
- A "loss" is a trial allocated to the true worse arm (if, at that moment, the observed better arm is in fact the true worse arm).

Optimal Resource Allocation for 2AB

Frantz: As N and n increase, it holds that:

$$N - n^* \approx e^{cn^*}$$

where N is the total number of available trials and n* is the optimal number of trials to be allocated to the worse observed arm A_L.

(Daniel Frantz: *Non-linearities in Genetic Adaptive Search*, PhD Dissertation, Ann Arbor, Michigan)

The optimal allocation of trials to the observed better arm, N-n*, should increase exponentially with respect to the number of trials allocated to the observed worse arm.

Interpretation of the Solution

- 2AB illustrates perfectly the **"exploration" vs. "exploitation"** problem faced by any adaptive system.
- GA: there are 3^m rather than 2 arms.
- The schema theorem states that, under its assumptions, the **GA embodies a version of the optimal 2AB strategy** above:
- *In successive generations of a GA, the number of chromosomes representing "**observed better**" schemata increases exponentially w.r.t. the trials allocated to worse observed schemata.*

The Genetic Algorithm vs. the Optimal 2AB Strategy

- Grefenstette e.a. discuss the fitness function:

$$f(x) = \begin{cases} 2 \text{ if } x \in 111*\ldots* \\ 1 \text{ if } x \in 0***\ldots* \\ 0 \text{ otherwise} \end{cases}$$

- **eval**(1*…*) = ½ (3 instances out of 4 give 0, one gives 2)
- **eval**(0*…*) = 1
- **eval**(1*…*, n) → **eval**(111*…*, n) ≈ 2 after some generations
- Instances of 111*…* - also instances of 1*…* - will be strongly selected, much more often than instances of 0*…*
  - Better and better approximation of the dynamic fitness only for **promising**, higher and higher order (sub-)schemata
  - **The GA follows the optimal allocation strategy for 2AB, but with a better complexity (speeds up the process).**

On-line Performance

- 2AB strategy is fit for **adaptation**, or equivalently, for **on-line performance**.
- Real-time control problems illustrate on-line performance: automatic control of machineries; learning to navigate in an unknown environment; predicting financial markets
- To **optimise** (find the best solution) vs. to **satisfice** (find a good enough solution with restricted resources). Herbert Simon.
  - approximate solution found without exceeding the existing resources.
  - **Satisficing** means **maximizing**, for a given amount of resources (time, samples), the amount of information about the sub-spaces likely to provide good solutions if sampled in the future. It is what Genetic Algorithms do best.
  - **Optimising** may require hybridisation, e.g.: a GA (≈global search) + a Hill-climber (≈local search).

Schema Approach: Drawbacks

- Explains how convergence happens, **if it happens**.
  - It only predicts the expected number of chromosomes instantiating schemata, not the overall behaviour of the algorithm
- Ignores the constructive effect of operators
  - How does a non-represented schema appear in a future generation?
  - Which new schemata are more likely to be discovered?
- How is the estimate of the average fitnesses of schemata built from individual fitnesses in one (successive) generation(s) ?
- What about situations where the observed fitness of a schema and its actual fitness *are **not** correlated*?
  - **Topology.** (Route to optimum through neighbourhoods)

Schema Approach: Drawbacks (2)

- What about the speed of convergence?
- Schemas are fit for the likely building blocks of *one-point crossover*. Other structures deal better with other operators.
- One-step process only: is it just a one-step-focused process (Markov) or multiple-step?
- Royal Road functions, built according to the Schema Theorem to favour Genetic Algorithms, was better solved by the Random Mutation HillClimber
  - Nevertheless, the explanation for this (hybridisation with random search) does not affect the validity of the Schema Theorem
- The Schema Theorem remains an extremely useful conceptual tool for understanding the way Genetic Algorithms work.